

MLOPS PORTFOLIO PROJECT · CORRECTED EDITION

Automated Support Ticket Classification

The 30-Day Build Guide, with every known defect fixed

Supersedes both earlier PDFs

Automated-Support-Ticket-Classification-MLOps-30-Day-Guide.pdf

Ticket-Triage-MLOps-30-Day-Guide.pdf

Windows / PowerShell · one commit per day

Package: `automated_support_ticket_classification`

Reflects repository state through Day 24

28 commits, pushed to origin/main

Revised 17 September 2026

Contents

Part 0 **Read this first.** Why this edition exists, which guide wins, and exactly where your repo stands today.

Part 1 **Errata.** Thirty-two verified defects across the two original guides, your repo and the data, each with root cause and fix. Twenty are resolved and marked as such.

Part 2 **Repair sprint.** Nine ordered steps to a clean baseline. Eight are done; only the pyproject tidy-up remains.

Part 3 **Days 14 to 30, corrected.** Days 14 to 24 are complete and annotated with what actually happened, including two defects found while building them. Day 25 onward is ahead of you.

Part 4 **Reference files.** Final, correct contents of every config file in one place.

Part 5 **Troubleshooting.** Expanded Windows table including the failures you actually hit.

Part 6 **Verification matrix.** What to run and what to expect, per day.

Part 7 **Interview preparation.** Including the new answers these fixes earned you.

How this edition differs

Every claim in Part 1 was reproduced on your machine before it was written down. Where a fix involves a version pin, the boundary version was checked against the actual package source rather than assumed. Nothing here is a guess; where something is cosmetic rather than broken, it says so.

Both blocking defects are resolved. The broken DVC install and the Git-owned data files would each have stopped you cold, on Day 15 and Day 17 respectively, with error messages that point nowhere useful. Part 1 keeps the full diagnosis of both, because the reasoning is more valuable than the fix and it is strong interview material.

Part 0 · Read this first

You were given the same guide twice

The two PDFs are not two guides. They are one 82-page guide published under two package names. A normalised line-by-line comparison of the extracted text found **zero** substantive differences in content: same 30 days, same code, same commands, same interview questions. The only variations are:

Item	Automated-Support-Ticket... PDF	Ticket-Triage PDF
Python package	<code>automated_support_ticket_classification</code>	<code>ticket_triage</code>
Repo / image name	<code>automated-support-ticket-classification</code>	<code>ticket-triage</code>
GitHub handle	hard-coded as <code>raviteja311</code>	left as <code><user></code> and <code><your-user></code>
Render URL	<code><YOUR_RENDER_URL></code>	<code>https://ticket-triage.onrender.com</code>

Decision, already made for you. Your repo is called `Automated-Support-Ticket-Classification` and your package is already `automated_support_ticket_classification`. That is the canonical naming for the rest of this document. Delete or archive the Ticket-Triage PDF so you never half-follow it and end up with `ModuleNotFoundError: No module named 'ticket_triage'`.

Where your repo actually stands

Measured, not assumed. **Day 24 is complete and pushed.** Every blocking defect found during the audit is resolved, and two further defects were found and fixed while building Days 16 to 24:

Signal	State	Detail
Progress	Day 24 done	28 commits on <code>main</code> , all pushed. Day 25 (CI) is next.
Test suite	13 passed	Data, model, script, and API tests, including cases the guide omits.
Lint and format	clean	<code>ruff check</code> passes; 18 files formatted. Day 25 CI will be green.
Pipeline	locked	Four DVC stages reproduced, <code>dvc.lock</code> committed, artifacts pushed to the remote.
Model	0.9300 / 0.9297	Accuracy and macro-F1 across five classes, matching the guide's published figures.
Service	containerised	FastAPI with <code>/</code> , <code>/health</code> , <code>/predict</code> ; 711 MB image; <code>\$PORT</code> override verified.
Data integrity	fixed	Five classes restored. The dataset had silently been four (D1 below).
Remaining defects	2 open	A3 and A9, both cosmetic one-line edits in <code>pyproject.toml</code> .
Day 25 onward	not started	No CI workflow, no <code>prometheus.yml</code> , no <code>render.yaml</code> .

Two defects found while building Days 16 to 24

Neither appears in the original audit, because neither was visible until the relevant day was built. Both are written up in full in Part 1.

ID	Found on	What it was
D1	Day 17, reading <code>dvc metrics show</code>	The dataset had four classes, not five. <code>shipping</code> was missing entirely and <code>account</code> was contaminated with the technical templates. Fixed in <code>ee2a709</code> ; accuracy rose from 0.9217 to 0.9300.

B16	Day 20, running <code>ruff check</code>	The guide's own <code>/predict</code> handler uses <code>zip()</code> without <code>strict=</code> , violating B905 from the bugbear ruleset the guide itself enables on Day 4. Following the guide literally produces a red CI job on Day 25.
------------	---	--

What is still open

Two defects remain. Both are one-line edits to `pyproject.toml` and neither blocks anything:

ID	Issue	Fix
A3	<code>description</code> sits in <code>[build-system]</code>	Move the line up under <code>[project]</code> .
A9	MLflow deprecation warnings on every test run	Add the <code>filterwarnings</code> block to <code>[tool.pytest.ini_options]</code> .

Severity key

CRITICAL stops you making progress. **HIGH** silently produces a wrong or unprofessional result. **MEDIUM** real but survivable. **COSMETIC** the guide says something untrue but nothing breaks. **FIXED** already resolved in your repo.

You are clear to start Day 25. Everything that would have stopped you is resolved: the broken DVC install, the Git-owned data files, the four-class dataset, and the lint violation that would have reddened your first CI run. Part 1 keeps the full diagnosis of each, because the reasoning is worth more than the fix and all of it is interview material.

Part 1 · Errata

Thirty-two defects. Twenty are fixed and marked **FIXED**; the diagnosis is kept because the reasoning is the valuable part. Group A is your repo, Group B the guides themselves (each tied to the day it detonates), Group C documentation that is simply wrong but harmless, and Group D defects found in the data and code while building Days 16 to 24.

Summary

ID	Severity	Where	One-line statement
A1	FIXED	tests/	A test imported <code>models/train.py</code> , a file that never existed.
A2	FIXED	models/	<code>models/__init__.py</code> turns an artifact directory into an importable package.
A3	MEDIUM	pyproject.toml	<code>description</code> sits in <code>[build-system]</code> instead of <code>[project]</code> .
A4	FIXED	tests/	Lint and format were failing, which would have made Day 25 CI red on arrival.
A5	FIXED	data/	Three CSVs are tracked by Git, which DVC refuses to take over.
A6	FIXED	.gitignore	<code>metrics/metrics.json</code> is ignored, defeating Day 12 and Day 16.
A7	FIXED	environment	DVC 3.55.2 is incompatible with the installed <code>pathspec 1.1.1</code> .
A8	FIXED	environment	scikit-learn and scipy disagree, warning on every model fit.
A9	COSMETIC	environment	Pydantic deprecation warnings from MLflow drown your test output.
A10	FIXED	environment	Python 3.12 locally, 3.11 in Docker and CI.
B1	FIXED	Day 2	The day ignores <code>data/</code> then tells you to <code>git add data</code> . It exits 1.
B2	HIGH	Day 3	<code>pip install -e .</code> runs before any <code>__init__.py</code> exists, so it installs nothing.
B3	MEDIUM	Day 3	PDF text reflow puts <code>name</code> and <code>description</code> in the wrong TOML tables.
B4	CRITICAL	Day 3	Pins 11 packages but not the 3 transitive ones that actually break the build.
B5	FIXED	Day 6	<code>Path("params.yaml")</code> is relative to the working directory.
B6	FIXED	Day 6	Pydantic v2 warns on every <code>model_</code> field name.
B7	FIXED	Day 10/11	Metrics stay as numpy scalars, which are not JSON serialisable.
B8	FIXED	Day 11	<code>mlflow</code> may be <code>None</code> where static analysis can see it.
B9	FIXED	Day 16	DVC stages omit <code>config.py</code> , so config edits do not trigger a rerun.
B10	FIXED	Day 20	The API cleans text differently from training. Classic train/serve skew.
B11	FIXED	Day 20/22	The empty-text branch is unreachable; whitespace-only input is the real gap.
B12	FIXED	Day 23	The lean image installs MLflow and DVC, contradicting Day 11's rationale.
B13	FIXED	Day 25	CI assumes the venv is <code>.venv</code> ; yours is <code>MLOps</code> , so ruff scans it.
B14	FIXED	Day 15	A Windows backslash path gets committed into <code>.dvc/config</code> .
B15	MEDIUM	Day 1/30	One guide bakes in a GitHub handle, the other leaves literal placeholders.
C1	COSMETIC	Day 4	Claims 1 file reformatted when the repo has no Python files yet.
C2	COSMETIC	Day 2	Claims 5 files changed, which is unreachable given B1.
C3	COSMETIC	Day 2	<code>.github/workflows/</code> is created but never committed.

C4	MEDIUM	whole guide	Two PDFs, two package names, one project. Root of all naming confusion.
B16	FIXED	Day 20	<code>zip()</code> without <code>strict=</code> violates B905, a rule the guide itself enables on Day 4.
D1	FIXED	data	The dataset had four classes, not five: <code>shipping</code> missing, <code>account</code> contaminated.

Group A · Defects found in your repository

Five of these six are now fixed and pushed. The diagnosis is kept in full because understanding why each one happened is worth more than the one-line fix, and every one of them is interview material.

A1 **FIXED** A test pointed at files that never existed

SYMPTOM. `pytest` reported `FileNotFoundError: ...\\models\\train.py` from `tests/test_model_scripts.py`.

ROOT CAUSE. The test loaded `models/train.py` and `models/evaluate.py` from the repo root by file path. Neither file exists and neither should. In this layout the code lives in `src/automated_support_ticket_classification/models/`, while root-level `models/` is the DVC output directory that holds `model.joblib` and nothing else. Worth knowing: **this test appears in neither PDF**. Both guides' test suites are exactly `test_data.py`, `test_model.py` and later `test_api.py`. The file was introduced in your commit `34913e6` and had never passed.

FIX. Keep the intent, drop the file-path loading:

```
from automated_support_ticket_classification.models.evaluate import main as evaluate_main
from automated_support_ticket_classification.models.train import main as train_main

def test_model_scripts_import_and_define_main():
    assert callable(train_main)
    assert callable(evaluate_main)
```

Already applied. The suite is 6 passed. See A2 for why the wrong paths looked plausible.

A2 **FIXED** `models/__init__.py` disguises an artifact directory as a package

SYMPTOM. None yet. It is why A1 looked reasonable when it was written.

ROOT CAUSE. Root `models/` exists only to receive `model.joblib`. An `__init__.py` was presumably added to keep the empty directory in Git, but it also makes `import models` resolve to it. Combined with `pythonpath = ["src"]` and `tests/__init__.py`, that is a live shadowing hazard and a misleading signal to any reviewer reading your tree.

FIX. Use the placeholder that means "keep this directory", not the one that means "this is a package":

```
git rm --cached models/__init__.py
rm models/__init__.py
New-Item -ItemType File models\\.gitkeep -Force
```

Applied in commit `48e07be`. `models/__init__.py` removed; `models/.gitkeep` retained.

A3 **MEDIUM** `description` is in the wrong TOML table

SYMPTOM. Your package has no description in its metadata, and `setuptools` sees an unexpected key under `[build-system]`.

ROOT CAUSE. Defect B3. When the PDF reflows the Day 3 code block, keys land out of order, and `description` ended up below the `[build-system]` header. Your file currently reads:

```
[build-system]
description = "Production-grade MLOps service that classifies customer support tickets."
requires = ["setuptools>=68"]
```

FIX. Move it up under `[project]`. Full corrected file in Part 4.

A4 **FIXED** Lint and format are already failing

SYMPTOM. Right now, in your repo:

```
> ruff format --check .
1 file would be reformatted, 13 files already formatted

> ruff check .
Found 5 errors. # includes I001 unsorted imports in tests/test_model.py
```

WHY IT MATTERS. Day 25 adds a CI job that runs `ruff check .` and `ruff format --check .`. The very first push after you create that workflow will go red, and you will spend the day debugging CI instead of learning it.

FIX. Clear it now, while it is two commands rather than a red badge:

```
ruff format .
ruff check . --fix
```

Applied in commit 2d10164 and 48e07be. `ruff check .` now reports All checks passed, and 13 files are formatted. The last 4 errors were E402s caused by the `sys.path` block in `train.py` and `evaluate.py`, removed with B5 already fixed properly in `config.py`.

A5 **FIXED** Your data CSVs are committed to Git

SYMPTOM. Nothing today. On Day 17, `dvc repro` fails.

ROOT CAUSE. The guide's Day 2 `.gitignore` ignores `data/`. Yours does not, so when you ran the generator and preprocessor the outputs were picked up by a `git add`. These three files are tracked:

```
data/raw/tickets.csv
data/processed/train.csv
data/processed/test.csv
```

DVC will not take ownership of a file Git already tracks. Reproduced on this machine with a minimal repo and the exact stage definition from Day 16:

```
ERROR: failed to reproduce 'generate': output 'data\raw\tickets.csv' is already
tracked by SCM (e.g. Git).
    You can remove it from Git, then add to DVC.
    To stop tracking from Git:
        git rm -r --cached 'data\raw\tickets.csv'
        git commit -m "stop tracking data\raw\tickets.csv"
```

FIX. Untrack the files and fix the ignore rules before Day 15. Full procedure in Repair 3 and Repair 4. The files stay on disk; only Git forgets them.

Applied in commit 48e07be. All three CSVs untracked, files untouched on disk. `git check-ignore` confirms they are now ignored. Day 17 is unblocked.

A6 **FIXED** `metrics/metrics.json` is ignored, and it must not be

SYMPTOM. Your evaluation output never appears in a commit or a PR diff.

ROOT CAUSE. Your `.gitignore` line 163 is `metrics/*.json`. Confirmed with `git check-ignore -v`. This directly contradicts two days of the guide: Day 12 says "commit metrics.json to Git so metric changes show up in PR diffs", and Day 16 declares the stage's metric with `cache: false`, which is DVC's way of saying *Git owns this file, not me*. With the current ignore rule, Day 16 produces a pipeline whose headline output vanishes.

FIX. Delete the `metrics/*.json` line, then commit the file. Keep `models/*.joblib` ignored; that one is correct.

Applied in commit 48e07be. `metrics/*.json` removed from `.gitignore` and `metrics/metrics.json` committed.

A7 **FIXED** DVC is broken in your environment

SYMPTOM. Every `dvc` subcommand that touches ignore rules, which is effectively all of them, dies with:

```
ERROR: unexpected error - cannot import name '_DIR_MARK' from  
'pathspec.patterns.gitwildmatch'
```

ROOT CAUSE. A transitive dependency moved a private symbol. `dvc 3.55.2` does this at `dvc/ignore.py:30`:

```
from pathspec.patterns.gitwildmatch import _DIR_MARK
```

In **pathspec 0.12.1**, `_DIR_MARK` is defined in `pathspec/patterns/gitwildmatch.py` at line 21, exactly where DVC looks. In **pathspec 1.1.1**, which is what you have installed, it moved to `pathspec/patterns/gitignore/spec.py` and the old module no longer re-exports it. Both locations were confirmed by reading the package source, not inferred from the traceback.

This is the hidden cost of defect B4: `requirements.txt` pins `dvc` exactly but says nothing about `pathspec`, so pip resolved the newest one and silently broke the pinned tool.

FIX. Pin the compatible pathspec and reinstall:

```
pip install "pathspec==0.12.1"
```

VERIFIED. With pathspec 0.12.1 on the path, the same minimal DVC repo that previously crashed on import ran its stage normally and progressed to the real, expected *already tracked by SCM* error from A5. The fix works.

Applied in commit 3407a18. `pathspec==0.12.1` installed and pinned in `requirements.txt` with a comment explaining why. `dvc init` and the local remote then completed normally on Day 15.

A8 **FIXED** The `iprint` warning on every model fit

SYMPTOM. Twice per test run:

```
sklearn\linear_model\_logistic.py:455: OptimizeWarning:  
Unknown solver options: iprint
```

ROOT CAUSE. scikit-learn passes a legacy verbosity option down to SciPy's L-BFGS-B solver. SciPy removed it. Tracing the boundary precisely, by reading each wheel's source rather than guessing:

Package version	Passes or accepts <code>iprint</code> ?
scikit-learn 1.5.2 (yours) through 1.9.1	Still sends it. Every release does.
scipy 1.14.1, 1.15.0, 1.16.0, 1.16.2, 1.17.0	Accepts it. No warning.
scipy 1.18.1 (yours)	Rejects it. Warning on every fit.

Upgrading scikit-learn does **not** help. This was worth checking, because the obvious instinct is to bump the ML library. The fix has to be on the SciPy side.

FIX. Pin SciPy below the change:

```
pip install "scipy<1.18"
```

The warning is genuinely harmless; the unknown option is ignored and the solver converges correctly. But the guide's entire premise is reproducible pinned environments, and an unpinned transitive dependency that changes behaviour is exactly what pinning is supposed to prevent. Pin it, and add SciPy and NumPy to `requirements.txt` so this class of surprise stops recurring.

Applied in commit f14d31e. `scipy<1.18` and `numpy<3` pinned; SciPy is now 1.17.1 and the warning is gone. The only warnings left in the suite are the MLflow ones from A9.

A9 **COSMETIC** Sixteen Pydantic warnings bury your test output

SYMPTOM. Every `pytest` run prints a wall of `PydanticDeprecatedSince20` warnings from `mlflow/gateway/config.py`.

ROOT CAUSE. Not your code. MLflow 2.18.0 still uses Pydantic v1-style `@validator` decorators, and Pydantic 2.9.2 warns about each one. Nothing you write will silence it at the source.

FIX. Filter it in `pyproject.toml` so real warnings from your own code stay visible instead of being lost in the noise:

```
[tool.pytest.ini_options]  
pythonpath = ["src"]  
testpaths = ["tests"]  
addopts = "-q"  
filterwarnings = [  
    "ignore::DeprecationWarning:mlflow.*",  
    "ignore:Pydantic V1 style.*:UserWarning",  
]
```

A10 **FIXED** Python 3.12 locally, 3.11 everywhere else

SYMPTOM. None yet. It surfaces as "works on my machine, fails in CI".

ROOT CAUSE. Your `MLops` venv is Python 3.12.10. The guide's Dockerfile uses `python:3.11-slim` and the CI workflow pins `python-version: "3.11"`. Three environments, two interpreters.

FIX. Pick one and say so everywhere. The lower-friction choice is to accept 3.12 and move Docker and CI up to match, since your venv already works:

- `pyproject.toml`: `requires-python = ">=3.11"` stays as is, it is a floor not a target.
- Dockerfile: `FROM python:3.12-slim` in both stages.
- `ci.yml`: `python-version: "3.12"`.

Whichever you choose, the rule to state in an interview is that the interpreter version is part of the environment contract and belongs in exactly one place conceptually, even if it is physically written in three files.

Applied in commit f14d31e. Settled on 3.12 to match the local interpreter. The Dockerfile uses `python:3.12-slim` in both stages; Day 25 should pin `python-version: "3.12"` to match.

Group B · Defects in the guides, listed by the day they detonate

B1 **HIGH** Day 2 ignores `data/`, then asks Git to add it

Day 2's `.gitignore` contains `data/` and `models/`. The same page then tells you to drop `.gitkeep` files into those directories and run:

```
git add .gitignore data models metrics
```

Reproduced in a scratch repo with exactly those contents:

```
exit 1
The following paths are ignored by one of your .gitignore files:
data
models
hint: Use -f if you really want to add them.

staged: .gitignore, metrics/.gitkeep      # 2 files, not 5
```

Git will not descend into an ignored directory, so a negation like `!data/raw/.gitkeep` does not rescue it either; the parent must not be ignored in the first place. **FIX.** Ignore the *contents*, not the directories:

```
# Data and model artifacts: DVC-tracked, not Git-tracked
/data/raw/*
/data/processed/*
!/data/raw/.gitkeep
!/data/processed/.gitkeep
/models/*
!/models/.gitkeep
# metrics/ is deliberately NOT ignored: Day 12 and Day 16 need it in Git
```

B2 **HIGH** Day 3 installs a package that does not exist yet

Day 3 runs `pip install -e .` and promises this expected output:

```
Successfully installed automated-support-ticket-classification-0.1.0 ...
# and: python -c "import automated_support_ticket_classification" prints nothing (success)
```

It cannot print nothing, because there is nothing to import. Day 2 created the directories but no `__init__.py`; the first one appears on Day 5. Confirmed directly:

```
>>> find_packages(where="src")  # src/automated_support_ticket_classification/ exists, empty
[]
```

The install succeeds and installs zero packages, so the failure is deferred to Day 5 and shows up as a confusing `ModuleNotFoundError` in a day that has nothing to do with packaging.

FIX. Create the package marker on Day 2, alongside the skeleton:

```
New-Item -ItemType File src\automated_support_ticket_classification\__init__.py -Force
```

Then Day 3's `pip install -e .` behaves as advertised. If you already passed Day 3, simply re-run `pip install -e .` now.

B3 MEDIUM Day 3's TOML block reflows into the wrong tables

In the PDF, the Day 3 `pyproject.toml` block renders with `name` detached from its table and floating to the bottom of the page, and `description` adjacent to the `[build-system]` header. Transcribe it literally and you get A3. This is a publishing defect rather than a code defect, but it is the reason a real error is sitting in your repo, so it earns a place on the list.

FIX. Ignore the PDF's layout for this block and use the canonical form in Part 4. General rule for the rest of the guide: whenever a code block looks like its lines are out of order, they probably are. Section headers such as `[project]` always come before their keys.

B4 CRITICAL The pins do not pin the things that break

Day 3 presents `requirements.txt` with eleven `==` pins and the claim that this makes builds reproducible. It does not, because none of the three packages that actually broke your environment are named in it:

Package	Pinned?	What went wrong
<code>pathspec</code>	No	Resolved to 1.1.1 and broke DVC outright (A7).
<code>scipy</code>	No	Resolved to 1.18.1 and warns on every fit (A8).
<code>numpy</code>	No	Floats freely; the next thing to surprise you.

FIX. Add the three pins. Better still, generate a full lock once the environment is known-good, which is what a real team does:

```
pip freeze > requirements.lock.txt
```

Keep the human-readable `requirements.txt` for intent and the lock file for exact reproduction. This is a strong interview point: pinning direct dependencies is not the same as pinning the dependency *graph*, and the difference is precisely the failure you hit.

B5 **HIGH** Day 6's config path is relative to the working directory

Day 6 writes `PARAMS_PATH = Path("params.yaml")`. That resolves against the process working directory, so the loader works only when you happen to be standing in the repo root:

```
FileNotFoundError: [Errno 2] No such file or directory: 'params.yaml'
```

It breaks when you run `uvicorn` from elsewhere, run `pytest` from a subdirectory, or load config inside a container with a different `WORKDIR`.

You already fixed this. Your `config.py` anchors to the package with `parents[2]` and resolves all four paths through a helper. That is the correct fix and it is carried into Part 4 as canonical. Worth noting for yourself: it is also what makes the Day 23 Dockerfile work, since `parents[2]` from `/app/src/automated_support_ticket_classification/config.py` lands exactly on `/app`.

B6 **MEDIUM** Pydantic v2 warns on every `model_` field

Day 6's `ModelConfig` declares `model_path`. Pydantic v2 reserves the `model_` prefix for its own API. Reproduced:

```
UserWarning: Field "model_path" in ModelConfig has conflict with protected namespace "model_".
```

Harmless until something runs with `-W error`, at which point config loading raises and every test fails at once with a message that has nothing to do with the test.

FIX. Declare the namespace open on each model. Your repo already does this; the guide never mentions it:

```
from pydantic import BaseModel, ConfigDict

class ModelConfig(BaseModel):
    model_config = ConfigDict(protected_namespaces=())
    ...
```

B7 **MEDIUM** Metrics stay as numpy scalars in train.py

Day 10 and Day 11 write `acc = accuracy_score(...)`, which returns `numpy.float64`, not a Python `float`. Day 12's `evaluate.py` casts with `float(...)`; `train.py` never does. MLflow tolerates the numpy type, so nothing fails today, but the moment you serialise those values you get:

```
TypeError: Object of type float64 is not JSON serializable
```

FIX. Cast at the boundary, in `train.py` as well as `evaluate.py`:

```
acc = float(accuracy_score(test_df["label"], preds))
f1 = float(f1_score(test_df["label"], preds, average="macro"))
```

Your repo already does this. Included so the corrected guide stays self-consistent.

B8 **COSMETIC** The optional-import guard is incomplete

Day 11 writes `if USE_MLFLOW:` then calls `mlflow.start_run()`. At runtime this is safe, because the `except ImportError` branch sets `USE_MLFLOW = False` in the same breath. A type checker cannot see that coupling and will flag a possible `None` attribute access.

FIX. Make the invariant explicit: `if USE_MLFLOW and mlflow is not None:`. Your repo already does. Listed as cosmetic because nothing actually breaks; it is a readability and static-analysis improvement, not a bug.

B9 **FIXED** Day 16's DVC stages under-declare their dependencies

Each stage lists only its own module, for example `src/.../models/train.py`. But every module also imports `config.py` and `logger.py`. Edit `config.py` and DVC will report the pipeline up to date, because no declared dependency hash changed. That is a silent reproducibility hole, and it is precisely the failure mode Day 16's own "Common mistakes" section warns about in the case of params.

FIX. Add the shared modules to every stage's `deps`. The corrected `dvc.yaml` in Part 4 does this.

Applied in commit 375b2b2. All four stages declare `config.py` and `logger.py`.

B10 **FIXED** The API preprocesses differently from training

Training normalises text through `clean_text`, which lowercases **and** collapses runs of whitespace:

```
def clean_text(text: str) -> str:
    text = text.lower()
    text = re.sub(r"\s+", " ", text)
    return text.strip()
```

Day 20's `/predict` handler does only `request.text.strip().lower()`. Two different transforms on the two sides of the same model. This is train/serve skew, textbook edition.

HONEST ASSESSMENT OF IMPACT. Small, in this specific case. TF-IDF's default tokenizer splits on non-word characters, so collapsing whitespace rarely changes the token stream. **WHY FIX IT ANYWAY.** The bug class is one of the most commonly probed in ML engineering interviews, the fix costs one import, and leaving two independent definitions of "clean" in a codebase guarantees they diverge the moment preprocessing grows.

FIX. Import the single definition rather than restating it:

```
from automated_support_ticket_classification.data.preprocess import clean_text
...
text = clean_text(request.text)
```

Applied in commit 678770b. The handler imports the single definition rather than restating it.

B11 **FIXED** The empty-text guard is unreachable, and the real gap is untested

Day 19 declares `text: str = Field(..., min_length=1)`. Day 20 then writes:

```
text = request.text.strip().lower()
if not text:
    raise HTTPException(status_code=422, detail="text must not be empty")
```

Pydantic rejects `""` with a 422 before the handler ever runs, so that branch never fires for the empty string. Day 22's `test_predict_rejects_empty_text` passes, but it is testing Pydantic, not your code, which is not what the guide claims it is testing.

The branch is reachable, for a case nobody tests: `" "` has length 3, so Pydantic admits it, and only the handler's `strip()` catches it.

FIX. Keep the guard, and add the test that actually exercises it:

```
def test_predict_rejects_whitespace_only_text():
    response = client.post("/predict", json={"text": " "})
    assert response.status_code == 422
```

Applied in commit 84431e1. Added, and the empty-text test is annotated as testing pydantic rather than the handler. Suite is 13 passed.

B12 **FIXED** The lean runtime image is not lean

Day 11 justifies the optional-import pattern by saying it lets the file "stay lean in Docker". Day 23's Dockerfile then does `pip install -r requirements.txt`, which installs `mlflow` and `dvc` into the runtime image, along with their considerable dependency trees. The env flag `DISABLE_MLFLOW=1` stops MLflow being *used*; it does nothing about it being *installed*. The stated rationale and the actual build contradict each other.

FIX. Add a third requirements file containing only what serving needs, and point the Dockerfile at it. This makes the Day 11 story true, shrinks the image substantially, and gives you a real answer to "how did you keep your production image small".

```
# requirements-serve.txt - runtime only, no MLflow, no DVC
scikit-learn==1.5.2
scipy<1.18
pandas==2.2.3
joblib==1.4.2
PyYAML==6.0.2
pydantic==2.9.2
fastapi==0.115.5
uvicorn[standard]==0.32.1
prometheus-fastapi-instrumentator==7.0.0
```

The model is still trained at image build time, and `train.py`'s `try: import mlflow / except ImportError` handles its absence exactly as designed. This is the first time that code path has been genuinely exercised.

Applied in commit f14d31e. `requirements-serve.txt` added and used by the runtime stage. Final image 711 MB; build context 3.26 kB once `MLOps` was added to `.dockerignore`.

B13 **HIGH** Day 25's CI assumes your venv is called `.venv`

Ruff excludes common virtual environment directory names by default, `.venv` among them. Yours is called `MLOps`, which ruff has never heard of, so `ruff check .` walks several thousand files of site-packages and reports other people's style violations as yours.

FIX. Already present in your `pyproject.toml`, which lists `MLOps` under `[tool.ruff] exclude`. The guide never mentions it, so if you rebuild the project from the PDF you will hit this again. It is included in the Part 4 reference file.

Related and worth doing: name the venv `.venv` next time. Every tool in the Python ecosystem special-cases that name. Fighting the default costs you a config line in every tool you add.

B14 **FIXED** Day 15 commits a Windows path into `.dvc/config`

`dvc remote add -d localremote ../dvc-storage` writes that backslash path into `.dvc/config`, which is committed to Git. It works on your machine and is meaningless on the Linux runner that will eventually try to use it.

FIX. Use a forward slash, which Windows accepts perfectly well: `dvc remote add -d localremote ../dvc-storage`. Day 18's own advice already points the right way: a local remote is for learning, and a real project uses a cloud remote so CI can pull.

Applied in commit 48e39b9. DVC stored the URL as `../../dvc-storage`, relative with forward slashes.

B15 **MEDIUM** The two guides handle your identity differently

The Automated-Support-Ticket PDF hard-codes the GitHub handle `raviteja311` into the Day 1 clone URL and the Day 26 and Day 30 CI badge URLs. The Ticket-Triage PDF leaves `<user>` placeholders in the same spots and warns you to replace them.

FIX. Confirm `raviteja311` is actually your handle, then use it consistently. If it is not, every badge and clone URL in the first PDF needs changing. Day 30's "Common mistakes" flags exactly this, which is worth reading before you publish the README rather than after.

Group C · Documentation that is simply wrong

ID	Where	What it claims	What actually happens
C1	Day 4	1 file reformatted, 4 files left unchanged	At Day 4 the repo contains no Python files at all. The first one arrives on Day 5. Expect 0 files reformatted.
C2	Day 2	5 files changed, 40 insertions(+)	Two files stage, and the <code>git add</code> exits 1. See B1.
C3	Day 2	Creates <code>.github/workflows/</code> as part of the skeleton	Git cannot track an empty directory and the day's <code>git add</code> list omits it, so it never reaches the remote. Harmless: Day 25 creates it again with a file in it.
C4	all	Two separate guides	One guide, two package names. Covered in Part 0.

Group D · Defects found while building Days 16 to 24

Neither of these was in the original audit, because neither was visible until the relevant day existed. Both are fixed. Both are better interview material than anything in Groups A to C, because you found them by reading output rather than by being told.

D1 **FIXED** The dataset had four classes, not five

HOW IT SURFACED. Reading `dvc metrics show` on Day 17. The per-class table listed `account`, `billing`, `general` and `technical`, with supports summing to exactly 600. `shipping` was absent.

ROOT CAUSE. A transcription error in `TEMPLATES` in `data/generate.py`, made when the generator was copied out of the PDF. The `"shipping": [` key sits at the bottom of page 22 and its five entries resume on page 23, with an unrelated file-path caption interleaved. Two things went wrong at that page break:

1. `shipping` was dropped entirely, leaving four classes.
2. The five `technical` templates were filed under `account`, giving it ten templates, half describing technical problems. Five templates that appear nowhere in either PDF had been written to fill the resulting gap.

So half of every `account` row was a technical ticket wearing the wrong label. The model was not underperforming; it was being asked to learn a contradiction. A quieter third symptom confirmed it: the `order` entry in `FILLERS` was dead code, because only the missing shipping templates ever used `{order}`.

EVIDENCE BEFORE THE FIX. `account` had the worst recall of any class at 0.869 and `technical` the worst precision at 0.905, which is precisely the signature of one class's examples being predicted as another.

FIX. Restored the canonical taxonomy: five classes, five templates each, `shipping` reinstated, technical templates moved back out of `account`.

Metric	Before (4 classes)	After (5 classes)
accuracy	0.9217	0.9300
f1_macro	0.9212	0.9297
account recall	0.869	0.920

Accuracy rose *despite* the task gaining a class, because removing the contradictory labels mattered more than the extra class cost. And `account` recall becoming unremarkable among its peers is exactly what you would predict if contamination had been the cause.

INDEPENDENT CONFIRMATION. 0.9300 and 0.9297 match the figures the guide prints as expected output for Day 10, so the dataset now matches the one the guide intends.

WHY NOTHING CAUGHT IT. The training logs happily reported a healthy 0.92. The unit tests assert on row count and column names, not on the label set. Only per-class metrics laid out side by side made it visible, which is the whole argument for Day 12 existing.

Applied in commit ee2a709. A guard test, `test_generate_covers_all_five_categories`, now pins the label set so this cannot regress silently. Written up as E2 in `docs/experiments.md`.

B16 **FIXED** The guide's own API code fails the guide's own lint rules

HOW IT SURFACED. Running `ruff check .` on Day 20, immediately after transcribing the `/predict` handler.

```
app.py:40:43: B905 `zip()` without an explicit `strict=` parameter
```

ROOT CAUSE. Day 20 writes `zip(model.classes_, proba)`. Day 4 configures ruff with `select = ["E", "F", "I", "UP", "B"]`, and `B` is bugbear, which includes B905. The guide's own code violates the ruleset the guide tells you to enable. Following it literally gives you a red CI job on Day 25, in a file you did not write yourself.

FIX. Add the parameter rather than suppressing the rule. It is also semantically correct: the two sequences must be the same length, and if they ever are not, silent truncation is the worst possible outcome.

```
scores = {cls: float(p) for cls, p in zip(model.classes_, proba, strict=True)}
```

Applied in commit 678770b. `ruff check .` passes across all 18 files, so Day 25's CI should be green on its first run.

The pattern worth noticing. D1 and A3 have the same cause: a PDF page break scrambling a code block during transcription. A3 put a TOML key in the wrong table, which was harmless. D1 dropped a class and mislabelled a quarter of the dataset, which was not. When a code block in a PDF spans a page boundary, check it against the surrounding structure rather than trusting the reading order.

Part 2 · Repair sprint

Status: eight of nine repairs are done and pushed. Only Repair 2, a pair of one-line edits to `pyproject.toml`, is outstanding, and it blocks nothing. The full sequence is kept below so you can rebuild from scratch and so the reasoning survives.

Nine steps, in this order, run from the repo root with the `MLOps` environment active. Each repair names the errata it closes.

Order matters. Repair 4 untracks files from Git. Do Repair 3 first so the ignore rules are in place, otherwise the next `git add -A` puts them straight back.

DONE Repair 1 · Fix the broken environment (closes A7, A8, B4)

```
pip install "pathspec==0.12.1" "scipy<1.18"
dvc --version          # must print 3.55.2 with no traceback
pytest -q              # the iprint OptimizeWarning should be gone
```

Then record the pins so this never resolves badly again. Add to `requirements.txt`:

```
# Transitive pins: without these, pip resolves versions that break dvc and sklearn
pathspec==0.12.1
scipy<1.18
numpy<3
```

Finally, capture the exact working graph for reproducibility:

```
pip freeze > requirements.lock.txt
```

Applied across commits 3407a18 and f14d31e. `pathspec==0.12.1`, `scipy<1.18` and `numpy<3` are pinned in `requirements.txt`. SciPy is 1.17.1 locally and the `OptimizeWarning` is gone. The lock file has not been generated yet; it remains optional.

TODO Repair 2 · Fix `pyproject.toml` (closes A3, A9, B13)

Replace the file with the canonical version from Part 4. Three changes: `description` moves from `[build-system]` to `[project]`, a `filterwarnings` block is added, and the `MLOps` ruff exclusion is kept and commented so its purpose is obvious.

```
pip install -e .          # re-install so the corrected metadata takes effect
```

DONE Repair 3 · Fix `.gitignore` (closes A6, B1)

Find the block near the end that currently reads:

```
# Generated model artifacts
models/*.joblib
metrics/*.json
```

Replace it with:

```
# Data and model artifacts: DVC-tracked, deliberately not in Git
/data/raw/*
/data/processed/*
!/data/raw/.gitkeep
!/data/processed/.gitkeep
/models/*
!/models/.gitkeep

# metrics/metrics.json is deliberately NOT ignored.
# Day 12 wants it in PR diffs; Day 16 declares it with cache: false,
# which means Git owns the file, not DVC.
```

DONE Repair 4 · Untrack the data artifacts (closes A5)

This is the one that unblocks Day 17. The files stay on disk; Git stops tracking them.

```
git rm -r --cached data/raw/tickets.csv data/processed/train.csv data/processed/test.csv
git status --short      # expect D for all three, and they still exist on disk
```

If you skip this, Day 17 fails with `output 'data\raw\tickets.csv' is already tracked by SCM (e.g. Git)` and you will lose the session working out why a brand-new DVC pipeline refuses to run.

DONE Repair 5 · Start tracking metrics.json (closes A6)

```
git add metrics/metrics.json
git check-ignore -v metrics/metrics.json    # must print nothing now
```

DONE Repair 6 · Demote models/ back to a plain directory (closes A2)

```
git rm --cached models/__init__.py
Remove-Item models\__init__.py
if (-not (Test-Path models\.gitkeep)) { New-Item -ItemType File models\.gitkeep }
```

DONE Repair 7 · Clear the lint and format backlog (closes A4)

```
ruff format .
ruff check . --fix
ruff check .          # must print: All checks passed!
ruff format --check .  # must print: N files already formatted
```

Do this before Day 25, not during it. Arriving at the CI day with a clean tree means the workflow goes green on its first run, which is the entire point of the exercise.

DONE Repair 8 · Commit the repair

On your repo this landed as two commits rather than one: `3407a18` for the pathspec pin and `48e07be` for the ignore rules, the untracking and the `sys.path` removal. The message below is the single-commit form if you are rebuilding from scratch.

```
git add -A
git status --short          # review carefully before committing

git commit -m "fix: correct packaging metadata, ignore rules and dependency pins

- move description from [build-system] to [project]
- ignore data artifacts, stop ignoring metrics.json
- untrack generated CSVs so DVC can own them
- pin pathspec and scipy to versions compatible with dvc and sklearn
- replace models/_init__.py with .gitkeep
- point test_model_scripts at the real package modules"
```

DONE Repair 9 · Verify the whole thing

Run all five. Every one must pass before you continue. All five were confirmed green on your repo before Days 13 to 15 were committed.

Command	Required result
<code>pytest</code>	<code>13 passed</code> , no <code>OptimizeWarning</code>
<code>ruff check .</code>	<code>All checks passed!</code>
<code>ruff format --check .</code>	no file would be reformatted
<code>dvc --version</code>	<code>3.55.2</code> , no traceback
<code>git ls-files data models</code>	only <code>.gitkeep</code> entries, no CSV, no joblib

Checkpoint, confirmed on your repo. All five checks pass: 13 tests, ruff clean, format clean, dvc 3.55.2 importable, and no artifacts tracked by Git. Days 13 through 24 are committed and pushed on top of that state. Day 25 is next.

What this repair is worth in an interview

Do not hide this work. It is the most senior-looking thing in the whole project, and it is real:

- **Dependency resolution.** "My pinned DVC broke because an unpinned transitive dependency moved a private symbol between minor versions. I traced it to the import in `dvc/ignore.py`, found the symbol had relocated in pathspec 1.x, pinned pathspec, and added a lock file so direct pins and graph pins stopped being the same claim."
- **Tooling boundaries.** "DVC refused to take over files Git already tracked. That is correct behaviour, not a bug: two version control systems cannot both own one file. I untracked them from Git and fixed the ignore rules that let them in."

- **Reading the right layer.** "The sklearn warning looked like an ML problem, so the instinct was to upgrade scikit-learn. I checked the source of every release from 1.5 to 1.9 and they all still send that option; the change was on the SciPy side. Pinning SciPy fixed it. Upgrading scikit-learn would not have."

Part 3 · Days 14 to 30, corrected

Each day keeps the original shape: objective, why it matters, files, how to run, expected output, commit. Where this edition departs from the PDFs, a **Corrected** note says exactly what changed and why. Days with no such note are unchanged from the original and were checked, not merely copied.

Day 14 · Run a real experiment

Modeling · DONE

Completed, commit 95276f9 . Both configurations were actually run. Unigrams and bigrams scored identically: accuracy 0.9217, macro-F1 0.9212. `ngram_max` stays at 1 and `docs/experiments.md` records the reasoning.

OBJECTIVE. Change one hyperparameter, retrain, compare runs in MLflow, and record the decision.

WHY IT MATTERS. This is the day the tracking setup pays for itself. You form a hypothesis, change exactly one variable, rerun, and compare. Then you write down the outcome and commit the decision, not just the code. An experiment you did not record is an experiment you will repeat.

FILES. Set `ngram_max: 2` in `params.yaml`, retrain, then create `docs/experiments.md` :

```
# Experiments log

| Run | ngram_max | accuracy | f1_macro | decision |
|-----|-----|-----|-----|-----|
| 1 | 1 | 0.9217 | 0.9212 | baseline |
| 2 | 2 | 0.9217 | 0.9212 | no change, reverted |

Conclusion: bigrams did not beat unigrams on this data, so we keep ngram_max=1.
```

```
# set ngram_max: 2 in params.yaml, then:
python -m automated_support_ticket_classification.models.train
mlflow ui # compare run 1 against run 2 at http://127.0.0.1:5000
# decide, write it up, then set ngram_max back to the winner
```

EXPECTED. Two runs side by side in MLflow. On your data bigrams gave no change at all, identical to four decimal places, so you keep `ngram_max: 1`.

The numbers above are the measured ones from your runs, not the guide's placeholders. The identical scores are a genuine result: on template-generated data the class signal sits in single content words, so phrase features have nothing to add. A portfolio experiments log with invented numbers is worse than none.

COMMIT. `chore: experiment with bigram features and record results`

Completed, commit 48e39b9 . DVC initialised and `localremote` set. DVC stored the URL as `../../dvc-storage` , relative with forward slashes, so B14 was avoided.

OBJECTIVE. Set up DVC for data and model versioning, pointed at a local remote.

WHY IT MATTERS. Git is bad at large binaries: it keeps the full history of every version of every blob, so the repo bloats and clones crawl. DVC keeps a tiny pointer file in Git and puts the bytes in a remote. This is how a team shares the exact dataset and model that produced a given commit.

Corrected. Two changes from the PDF. First, confirm Repair 1 and Repair 4 are done, or nothing on this day works. Second, use a forward slash for the remote path (B14) so the committed `.dvc/config` is not Windows-only.

```
# Preflight: both of these must be clean before you start
dvc --version                # 3.55.2, no traceback (Repair 1)
git ls-files data            # only .gitkeep entries (Repair 4)

dvc init                    # creates .dvc/ and .dvcignore
mkdir ../dvc-storage
dvc remote add -d localremote ../dvc-storage
git add .dvc .dvcignore .gitignore
```

EXPECTED.

```
Initialized DVC repository.
Setting 'localremote' as a default remote.
```

COMMON MISTAKES. Trying to `git add` the data file itself; you add DVC's pointer and DVC handles the bytes. Confusing `dvc add` , which is for external files you did not produce, with pipeline stages, which are for files your code produces. We use stages, starting tomorrow.

COMMIT. `chore: initialize DVC with a local remote`

Completed, commit 375b2b2 . `dvc dag` renders the correct four-node graph.

OBJECTIVE. Describe the whole workflow as a reproducible DAG in `dvc.yaml` .

WHY IT MATTERS. `dvc.yaml` declares each stage's command, its dependencies, and its outputs. DVC builds a dependency graph from that, so it knows exactly what must rerun when something changes. Four ad-hoc scripts become one reproducible pipeline.

Corrected. Every stage now also depends on `config.py` and `logger.py` (B9). In the PDF version, editing `config.py` leaves DVC convinced the pipeline is up to date, which is a silent reproducibility hole.

```

stages:
  generate:
    cmd: python -m automated_support_ticket_classification.data.generate
    deps:
      - src/automated_support_ticket_classification/data/generate.py
      - src/automated_support_ticket_classification/config.py
      - src/automated_support_ticket_classification/logger.py
    params:
      - data.n_samples
      - data.random_state
    outs:
      - data/raw/tickets.csv

  preprocess:
    cmd: python -m automated_support_ticket_classification.data.preprocess
    deps:
      - src/automated_support_ticket_classification/data/preprocess.py
      - src/automated_support_ticket_classification/config.py
      - src/automated_support_ticket_classification/logger.py
      - data/raw/tickets.csv
    params:
      - data.test_size
      - data.random_state
    outs:
      - data/processed/train.csv
      - data/processed/test.csv

  train:
    cmd: python -m automated_support_ticket_classification.models.train
    deps:
      - src/automated_support_ticket_classification/models/train.py
      - src/automated_support_ticket_classification/config.py
      - src/automated_support_ticket_classification/logger.py
      - data/processed/train.csv
      - data/processed/test.csv
    params:
      - model.max_features
      - model.ngram_max
      - model.C
      - model.max_iter
    outs:
      - models/model.joblib

  evaluate:
    cmd: python -m automated_support_ticket_classification.models.evaluate
    deps:
      - src/automated_support_ticket_classification/models/evaluate.py
      - src/automated_support_ticket_classification/config.py
      - src/automated_support_ticket_classification/logger.py
      - models/model.joblib
      - data/processed/test.csv
    metrics:
      - metrics/metrics.json:
          cache: false

```

```
dvc dag      # visualise the graph
```

EXPECTED. A four-node chain: generate, then preprocess, then train, then evaluate.

CONCEPT TO KEEP. `cache: false` on the metric means Git owns that file rather than DVC. That is the whole reason Repair 3 removed `metrics/*.json` from your `.gitignore`: an ignored file cannot appear in a PR diff, which is the only reason to declare it this way.

COMMIT. `feat: define DVC pipeline (generate to evaluate)`

Day 17 · Reproduce and lock the pipeline

Versioning · DONE

Completed, commit 55f45b0 . All four stages ran. `metrics.json` came out byte-identical to the committed copy, confirming the seeded generator and split are deterministic. Reading this day's output is how defect D1 was found.

OBJECTIVE. Run the pipeline with `dvc repro` and commit the lock file.

WHY IT MATTERS. `dvc repro` walks the DAG, skips stages whose inputs are unchanged, and writes `dvc.lock` : a record of the exact content hashes that produced these outputs. Committing the lock file is what lets anyone reproduce your exact result.

```
dvc repro
dvc metrics show
git add dvc.yaml dvc.lock .gitignore metrics/metrics.json
```

EXPECTED.

```
Running stage 'generate'...
Running stage 'preprocess'...
Running stage 'train'...
Running stage 'evaluate'...

Path          accuracy  f1_macro
metrics.json  0.93      0.9297
```

If you see this instead, Repair 4 was skipped:

```
ERROR: failed to reproduce 'generate': output 'data\raw\tickets.csv' is already
tracked by SCM (e.g. Git).
```

Go back and run Repair 4. DVC is telling you the truth: two version control systems cannot both own the same file.

COMMIT. `chore: run pipeline and lock reproducible outputs`

Completed, no commit. 4 files pushed; cache and remote in sync. This day changes nothing in the repo, so there is no commit to make. The guide says these are genuine commits and never filler, so do not manufacture an empty one.

OBJECTIVE. Upload data and model to the DVC remote, and understand pull.

WHY IT MATTERS. Git holds code and pointers; `dvc push` moves the heavy bytes. A teammate, your CI, or future you runs `git pull` then `dvc pull` to get exactly the data and model matching that commit. This clean split between code and artifacts is the core of the discipline.

```
dvc push
dvc status
dvc status -c      # compare workspace against the remote
```

EXPECTED. 3 files pushed, then Data and pipelines are up to date.

COMMON MISTAKES. Expecting `git push` to move data: it does not, ever. Using a local remote and then wondering why CI cannot pull: a folder on your laptop is not reachable from a GitHub runner. For a real project, point the remote at S3 or GCS, which is a one-command change and a good thing to be able to say you understand.

COMMIT. chore: push data and model artifacts to DVC remote

Completed, commit [84665b7](#) . Verified directly: pydantic rejects `""` but admits `" "`, which is exactly the asymmetry B11 describes.

OBJECTIVE. Define the pydantic models that form the API's contract.

WHY IT MATTERS. Schemas are the contract at your service boundary. Pydantic validates every incoming request automatically and documents the response shape, and FastAPI turns both into interactive docs for free. Validate at the edge and the rest of your code can trust its inputs.

FILES. `src/automated_support_ticket_classification/api/__init__.py` (empty) and `api/schemas.py`:

```
from pydantic import BaseModel, Field

class TicketRequest(BaseModel):
    text: str = Field(..., min_length=1, examples=["I was charged twice this month"])

class TicketResponse(BaseModel):
    label: str
    confidence: float
    all_scores: dict[str, float]
```

```
python -c "from automated_support_ticket_classification.api.schemas import TicketRequest;
print(TicketRequest(text='hi'))"
```

EXPECTED. `text='hi'`

Know what this does and does not catch. `min_length=1` rejects `""`. It admits `" "`, which has length 3. Day 20 handles that case, and Day 22 tests it. See B11.

COMMIT. `feat: add API request/response schemas`

OBJECTIVE. Build the app with `/health` and `/predict`.

WHY IT MATTERS. This is where the model becomes a product. The model loads lazily and is cached, so it is read from disk once rather than per request, and a `/health` endpoint lets platforms and monitors decide whether this instance should get traffic.

Completed, commit 678770b. Verified against the running service: a billing ticket scored 0.938, and all five classes appear in `all_scores`.

Corrected. Two changes. The handler reuses `clean_text` from the preprocessing module instead of its own `.strip().lower()` (B10). And `zip()` takes `strict=True` (B16): the guide's version omits it and trips B905 from the bugbear ruleset the guide itself enables on Day 4, which would redden your first CI run on Day 25.

```
from functools import lru_cache

import joblib
from fastapi import FastAPI, HTTPException

from automated_support_ticket_classification.api.schemas import TicketRequest, TicketResponse
from automated_support_ticket_classification.config import load_config
from automated_support_ticket_classification.data.preprocess import clean_text
from automated_support_ticket_classification.logger import get_logger

logger = get_logger(__name__)

app = FastAPI(title="Automated Support Ticket Classification API", version="1.0.0")

@lru_cache
def get_model():
    cfg = load_config()
    logger.info("Loading model from %s", cfg.model.model_path)
    return joblib.load(cfg.model.model_path)

@app.get("/health")
def health() -> dict:
    return {"status": "ok"}

@app.post("/predict", response_model=TicketResponse)
def predict(request: TicketRequest) -> TicketResponse:
    # Same normalisation as training. Do not restate it here.
    text = clean_text(request.text)
    if not text:
        raise HTTPException(status_code=422, detail="text must not be empty")
    model = get_model()
    proba = model.predict_proba([text])[0]
    # strict=True: the two sequences must be the same length (B16).
    scores = {cls: float(p) for cls, p in zip(model.classes_, proba, strict=True)}
    best = max(scores, key=scores.get)
    return TicketResponse(label=best, confidence=scores[best], all_scores=scores)
```

```
uvicorn automated_support_ticket_classification.api.app:app --reload
# then open http://127.0.0.1:8000/docs
```

EXPECTED. INFO: Uvicorn running on `http://127.0.0.1:8000`, and `/predict` returning `{"label": "billing", "confidence": 0.84, "all_scores": {...}}`.

COMMON MISTAKES. Loading the model at module import, so a missing file stops the whole app from starting. Reloading it per request, so the API is needlessly slow. `lru_cache` on a lazy getter solves both at once.

COMMIT. `feat: add FastAPI service with /health and /predict`

Completed, commit `1c8af5d` . Root endpoint returns the service name and links to `/docs` and `/health` .

OBJECTIVE. Add a friendly root endpoint and document the Windows run path.

WHY IT MATTERS. Small polish with outsized return. A root path that points at the docs instead of returning 404, and a README section someone can actually copy and paste.

```
@app.get("/")
def root() -> dict:
    return {
        "service": "automated-support-ticket-classification",
        "docs": "/docs",
        "health": "/health",
    }
```

```
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/predict `
  -ContentType "application/json" -Body '{"text":"my invoice is wrong"}'
```

EXPECTED.

```
label      confidence  all_scores
-----
billing    0.79        @{billing=0.79; ...}
```

COMMON MISTAKES. Reaching for `curl` in PowerShell and losing ten minutes to quoting. `Invoke-RestMethod` is the native tool and parses the JSON for you. Documenting a command you never ran.

COMMIT. `feat: add root endpoint and document local run`

OBJECTIVE. Test the endpoints, using a fixture that guarantees a model exists.

WHY IT MATTERS. API tests must run anywhere, including a CI runner where no model was ever trained. A session-scoped fixture builds a tiny one on demand, so the suite is self-sufficient. FastAPI's TestClient dispatches into the app without a server.

Completed, commit [84431e1](#) . Suite is **13 passed**. Verified the fixture works by moving `model.joblib` aside and re-running: the API tests passed against a model built from nothing, which is the CI condition.

Corrected. Three tests beyond the guide. `test_predict_rejects_whitespace_only_text` (B11) exercises the handler's guard, which the empty-string test never reaches.

`test_predict_scores_cover_every_class_and_sum_to_one` would have caught defect D1.

`test_root_points_at_the_docs` covers the Day 21 endpoint the guide adds but never tests.

FILES. `tests/conftest.py` :

```
from pathlib import Path

import joblib
import pandas as pd
import pytest

from automated_support_ticket_classification.config import load_config
from automated_support_ticket_classification.models.train import build_pipeline

@pytest.fixture(scope="session", autouse=True)
def ensure_model():
    """Guarantee a model file exists so API tests can load it (local and CI)."""
    cfg = load_config()
    path = Path(cfg.model.model_path)
    if not path.exists():
        df = pd.DataFrame(
            {
                "text": (
                    ["refund my payment invoice"] * 10
                    + ["app crashes with an error"] * 10
                    + ["reset my password please"] * 10
                    + ["where is my order shipping"] * 10
                    + ["what are your support hours"] * 10
                ),
                "label": (
                    ["billing"] * 10
                    + ["technical"] * 10
                    + ["account"] * 10
                    + ["shipping"] * 10
                    + ["general"] * 10
                ),
            }
        )
        pipe = build_pipeline(cfg)
        pipe.fit(df["text"], df["label"])
        path.parent.mkdir(parents=True, exist_ok=True)
        joblib.dump(pipe, path)
    yield
```

`tests/test_api.py` :

```

from fastapi.testclient import TestClient

from automated_support_ticket_classification.api.app import app

client = TestClient(app)

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "ok"

def test_predict_returns_label_and_confidence():
    response = client.post("/predict", json={"text": "I want a refund for a double charge"})
    assert response.status_code == 200
    body = response.json()
    assert "label" in body
    assert 0.0 <= body["confidence"] <= 1.0

def test_predict_rejects_empty_text():
    # Rejected by pydantic's min_length=1, before the handler runs.
    response = client.post("/predict", json={"text": ""})
    assert response.status_code == 422

def test_predict_rejects_whitespace_only_text():
    # Admitted by pydantic (length 3), rejected by the handler. This is the real guard.
    response = client.post("/predict", json={"text": "   "})
    assert response.status_code == 422

```

EXPECTED. 13 passed

COMMON MISTAKES. API tests failing in CI because no model file exists; the fixture is the answer. Forgetting `httpx` in the dev requirements, which `TestClient` needs.

COMMIT. test: add API tests with a model fixture

Completed, commit [f14d31e](#) . Image builds at 711 MB with a 3.26 kB build context. The model bakes in at `accuracy=0.9300` , identical to the local pipeline.

OBJECTIVE. Package the service into a lean, self-contained image.

WHY IT MATTERS. Docker makes the app run identically everywhere. A multi-stage build installs dependencies in one stage and copies only the result into a slim runtime. We bake the model in at build time so the image needs no external storage.

Corrected. Two changes. The runtime now installs `requirements-serve.txt` , which excludes MLflow and DVC (B12); the PDF's version installed both into an image it called lean, contradicting Day 11's stated rationale. And the base image moves to 3.12 to match your interpreter (A10); use 3.11 in both stages instead if you would rather move the other way.

```
# ----- Stage 1: build dependencies -----
FROM python:3.12-slim AS builder
WORKDIR /app
ENV PIP_NO_CACHE_DIR=1 PYTHONDONTWRITEBYTECODE=1
COPY requirements-serve.txt .
RUN pip install --prefix=/install -r requirements-serve.txt

# ----- Stage 2: lean runtime -----
FROM python:3.12-slim AS runtime
WORKDIR /app
ENV PYTHONUNBUFFERED=1 PYTHONPATH=/app/src DISABLE_MLFLOW=1
COPY --from=builder /install /usr/local
COPY src/ ./src/
COPY params.yaml ./

# Bake the model into the image: fast to train, keeps the image self-contained
RUN python -m automated_support_ticket_classification.data.generate \
    && python -m automated_support_ticket_classification.data.preprocess \
    && python -m automated_support_ticket_classification.models.train

EXPOSE 8000
CMD ["sh", "-c", "uvicorn automated_support_ticket_classification.api.app:app --host 0.0.0.0 --port ${PORT:-8000}"]
```

`.dockerignore` :

```
.git
.gitignore
.venv
venv
MLOps
.github
__pycache__
*.pyc
data/
models/
mlruns/
metrics/
tests/
docs/
notebooks/
.dvc/cache
.pytest_cache
.ruff_cache
README.md
```

```
docker build -t automated-support-ticket-classification .
```

EXPECTED. => exporting to image ... DONE

Why this build works at all is worth understanding, because it depends on the config fix from B5. Inside the image, `config.py` sits at `/app/src/automated_support_ticket_classification/config.py`, so its `parents[2]` resolves to `/app`, where `params.yaml` was copied. With the PDF's original CWD-relative path this is luck rather than design, and it stops being lucky the moment `WORKDIR` changes.

COMMON MISTAKES. No `.dockerignore`, so your `venv` and data get copied in and the build is slow and fat. Note the added `MLOps` line: without it your entire virtual environment goes into the build context. Binding `uvicorn` to `127.0.0.1` inside a container, which makes it unreachable from outside; it must be `0.0.0.0`.

COMMIT. build: add multi-stage Dockerfile

Day 24 · Run the container and add compose

Containers/CI · DONE

Completed, commit 9f972a7 . Container serves `/` , `/health` , `/docs` and `/predict` . The `$PORT` override was verified with `-e PORT=9001` , which is what Render needs on Day 28.

OBJECTIVE. Run the image locally and add a compose file for one-command startup.

WHY IT MATTERS. Running the container proves the image works before it reaches the cloud, which is a much slower place to debug. Compose codifies how the app is run and will host Prometheus on Day 27.

```
services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DISABLE_MLFLOW=1
```

```
docker run -p 8000:8000 automated-support-ticket-classification
# or:
docker compose up --build
# then, in another terminal:
Invoke-RestMethod http://127.0.0.1:8000/health
```

EXPECTED. `status / ok`

COMMON MISTAKES. Omitting `-p 8000:8000` , so the container runs happily and nothing can reach it. Port 8000 already taken: map a different host port with `-p 8001:8000` .

COMMIT. `build: add docker-compose for local run`

OBJECTIVE. Lint and test on every push and pull request.

WHY IT MATTERS. CI runs your quality gates on someone else's machine, automatically, so broken code never reaches main unnoticed. This one file is often what most impresses a reviewer, because it shows you think about correctness continuously rather than once.

Corrected. Three changes. Python 3.12 to match your environment (A10); `ruff format --check` now runs before `ruff check` so the cheaper, more common failure reports first; and a note about the `MLOps` exclusion (B13), which lives in `pyproject.toml` and is what stops ruff linting your whole virtual environment.

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements-dev.txt
          pip install -e .
      - name: Format check
        run: ruff format --check .
      - name: Lint
        run: ruff check .
      - name: Tests
        run: pytest
```

```
git add .github/workflows/ci.yml
git commit -m "ci: add lint and test workflow"
git push
```

EXPECTED. `lint-and-test ... completed`, and `All checks have passed`.

Your tree is ready for this day. `ruff check .` and `ruff format --check .` both pass across 18 files, and `pytest` is 13 passed, so the workflow should go green on its first push. That has only been true since the B16 fix on Day 20. Green locally is the only reliable predictor of green in CI, which is the real lesson of this day.

COMMIT. `ci: add lint and test workflow`

OBJECTIVE. Extend CI to build the image, and put a status badge on the README.

WHY IT MATTERS. The Docker job proves the image builds on a clean machine, not just yours. The badge is a public, green, permanently visible signal on your README.

```
docker-build:
  runs-on: ubuntu-latest
  needs: lint-and-test
  steps:
    - uses: actions/checkout@v4
    - name: Build Docker image
      run: docker build -t automated-support-ticket-classification:ci .
```

README badge (confirm the handle is yours before you commit it, see B15):

```
[[CI]](https://github.com/raviteja311/automated-support-ticket-classification/actions/workflows/ci.yml/badge.svg)]
(https://github.com/raviteja311/automated-support-ticket-classification/actions)
```

EXPECTED. Two green jobs, with `docker-build` waiting on `lint-and-test`.

INTERVIEW ANGLE. Order CI jobs with `needs:` so cheap checks run first and expensive ones only run on already-passing code. Lint takes seconds; a Docker build that trains a model takes minutes.

COMMIT. `ci: build docker image in pipeline and add status badge`

OBJECTIVE. Expose `/metrics` and add Prometheus to the local stack.

WHY IT MATTERS. You cannot operate what you cannot see. The instrumentator adds request counts, latencies and error rates at `/metrics`; Prometheus scrapes and stores them. This is the observability half of MLOps, and the same signals you would later chart in Grafana or alert on.

In `api/app.py`, add the import and one line after `app` is created:

```
from prometheus_fastapi_instrumentator import Instrumentator

app = FastAPI(title="Automated Support Ticket Classification API", version="1.0.0")
Instrumentator().instrument(app).expose(app)
```

`prometheus.yml`:

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: automated-support-ticket-classification
    metrics_path: /metrics
    static_configs:
      - targets: ["api:8000"]
```

`docker-compose.yml`, replaced:

```
services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DISABLE_MLFLOW=1

  prometheus:
    image: prom/prometheus:latest
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
    depends_on:
      - api
```

```
docker compose up --build
# hit the API a few times, then open:
# http://127.0.0.1:8000/metrics (raw)
# http://127.0.0.1:9090 (Prometheus UI)
```

EXPECTED. Counters such as `http_requests_total{handler="/predict",status="2xx"} 5.0`.

COMMON MISTAKES. Expecting metrics before any traffic: counters start at zero, so send a few requests first. Targeting `localhost` in `prometheus.yml` instead of the compose service name `api`; inside the compose network, `localhost` means the Prometheus container itself.

INTERVIEW ANGLE. Track the RED metrics: Rate, Errors, Duration. For an ML service specifically you would add input and prediction drift on top, which is operational monitoring plus model monitoring, and knowing they are two different jobs is the point.

COMMIT. `feat: expose Prometheus metrics and add monitoring stack`

OBJECTIVE. Deploy the container and get a public HTTPS URL.

WHY IT MATTERS. This is where the project becomes real: a live URL you can put on a resume. Render builds your Dockerfile, injects a `$PORT`, and polls `/health` before routing traffic to the instance.

`render.yaml`:

```
services:
- type: web
  name: automated-support-ticket-classification
  runtime: docker
  plan: free
  healthCheckPath: /health
  autoDeploy: true
```

ONE-TIME DASHBOARD SETUP.

1. Push everything to GitHub on `main`.
2. Go to the Render dashboard, then New, then Web Service.
3. Connect the `automated-support-ticket-classification` repo.
4. Render auto-detects the Dockerfile. Set Runtime: Docker, Health Check Path: `/health`, Instance Type: Free.
5. Create Web Service and wait for the build.
6. Open the URL Render gives you and append `/docs`.

EXPECTED. The service is live at the URL Render provides, and `/docs` runs a prediction against the cloud.

COMMON MISTAKES. Hard-coding port 8000 in the start command; Render injects `$PORT` and our `CMD` respects it. Panicking at a slow first request: free instances sleep when idle, and a cold start is expected, not broken.

Because the model is baked into the image, there is nothing else to configure: no external storage, no secrets for a basic deploy. That is a deliberate trade. The honest version to give in an interview is that baking the model in makes the image self-contained and the deploy trivial, at the cost of needing a rebuild to ship a new model. A model registry is the alternative, and it is on the future-improvements list for exactly this reason.

COMMIT. `deploy: add Render blueprint`

OBJECTIVE. Auto-deploy on merge, plus a scheduled smoke test of the live service.

WHY IT MATTERS. With auto-deploy on, every merge to main ships: that is continuous deployment rather than delivery. A smoke test then verifies the live URL is genuinely healthy, on a schedule and on demand, so you find out about an outage before a recruiter does.

```
name: Smoke Test

on:
  workflow_dispatch:      # run manually from the Actions tab
  schedule:
    - cron: "0 8 * * *"   # daily at 08:00 UTC

jobs:
  smoke:
    runs-on: ubuntu-latest
    steps:
      - name: Check live /health
        run: |
          url="${{ secrets.SERVICE_URL }}"
          code=$(curl -s -o /dev/null -w "%{http_code}" "$url/health")
          echo "Health status code: $code"
          test "$code" = "200"
```

```
# 1. In Render, Auto-Deploy is already on via render.yaml.
# 2. In GitHub: Settings -> Secrets and variables -> Actions, add
#    SERVICE_URL = https://your-service.onrender.com
# 3. Push, then run Smoke Test manually from the Actions tab.
```

EXPECTED. `Health status code: 200`

COMMON MISTAKES. Putting the live URL in the YAML instead of a secret. Scheduling the smoke test so often that it keeps waking a sleeping free instance; daily is plenty.

INTERVIEW ANGLE. Continuous delivery keeps main always releasable with a manual go; continuous deployment ships every passing merge automatically. Know your rollback path too: on Render, redeploy the previous successful build.

COMMIT. `ci: add scheduled smoke test against live service`

Day 30 · Finalize README and tag a release

Polish

OBJECTIVE. Polish the README, add CONTRIBUTING, cut `v1.0.0`.

WHY IT MATTERS. The last fifteen minutes are the highest-leverage of the whole month. A recruiter reads your README, not your source. A clear one with the live link, architecture at a glance, and a quickstart, plus a tagged release, reads as a finished project rather than an abandoned one.

Corrected. Replace every placeholder before committing (B15). The Automated-Support-Ticket PDF has `raviteja311` baked into the badge URLs, and the Ticket-Triage PDF has literal `<user>`. Search the finished README for `<` and for the handle, and confirm both are right. Add a line about the experiments log too: Day 14's `docs/experiments.md` is evidence you ran a real experiment, and it deserves a link.

```

# Automated Support Ticket Classification

[![CI](https://github.com/<you>/automated-support-ticket-classification/actions/workflows/ci.yml/badge.svg)]
(https://github.com/<you>/automated-support-ticket-classification/actions)
![Python](https://img.shields.io/badge/python-3.12-blue)
![License](https://img.shields.io/badge/license-MIT-green)

**Live demo:** <YOUR_RENDER_URL>/docs

Production-grade MLOps service that classifies customer support tickets into
billing, technical, account, shipping, and general categories.

## Architecture
Data generation -> preprocessing -> training (MLflow) -> evaluation ->
FastAPI service -> Docker -> GitHub Actions CI/CD -> Render -> Prometheus.

## Tech stack
Python, scikit-learn, DVC, MLflow, FastAPI, Docker, GitHub Actions,
pytest, ruff, Prometheus, Render.

## Quickstart
python -m venv .venv
.venv\Scripts\Activate.ps1      # Windows PowerShell
pip install -r requirements-dev.txt
pip install -e .
dvc repro                      # build data + model
uvicorn automated_support_ticket_classification.api.app:app --reload

Open http://127.0.0.1:8000/docs

## Reproduce the pipeline
dvc repro          # generate -> preprocess -> train -> evaluate
dvc metrics show  # accuracy / f1

## Tests
pytest

## Docker
docker build -t automated-support-ticket-classification .
docker run -p 8000:8000 automated-support-ticket-classification

## Experiments
See docs/experiments.md for the unigram vs bigram comparison and the decision.

## Future improvements
Transformer model, model registry, drift monitoring (Evidently),
Kubernetes deployment, Terraform IaC.

## License
MIT

```

```

git add README.md CONTRIBUTING.md
git commit -m "docs: finalize README and tag v1.0.0"
git push
git tag -a v1.0.0 -m "First production release"
git push origin v1.0.0

```

EXPECTED. Tag `v1.0.0` pushed, a green CI badge, and a live demo link.

COMMIT. `docs: finalize README and tag v1.0.0`

Part 4 · Reference files

Final, correct contents. Where your repo already matches, it says so. These supersede the equivalent blocks in both PDFs.

pyproject.toml (fixes A3, A9, B3, B13)

```
[project]
name = "automated_support_ticket_classification"
version = "0.1.0"
description = "Production-grade MLOps service that classifies customer support tickets."
requires-python = ">=3.11"

[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[tool.setuptools.packages.find]
where = ["src"]

[tool.ruff]
line-length = 100
target-version = "py311"
# MLOps is this project's virtual environment. Ruff only auto-excludes
# conventional names such as .venv, so it must be listed explicitly.
exclude = [
    ".git", ".venv", "venv", "MLOps",
    "__pycache__", ".pytest_cache", ".mypy_cache",
]

[tool.ruff.lint]
# E/W pycodestyle, F pyflakes, I isort, UP pyupgrade, B bugbear
select = ["E", "F", "I", "UP", "B"]

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]
addopts = "-q"
# MLflow 2.18 still uses pydantic v1 validators. Not our code, and it
# buries real warnings under 16 lines of noise on every run.
filterwarnings = [
    "ignore::DeprecationWarning:mlflow.*",
    "ignore:Pydantic V1 style.*:UserWarning",
]
```

config.py (fixes B5, B6; already correct in your repo)

```
from pathlib import Path

import yaml
from pydantic import BaseModel, ConfigDict

# Anchor to the package, never to the working directory.
# config.py -> ../src// -> ../src/ -> project root
PROJECT_ROOT = Path(__file__).resolve().parents[2]
PARAMS_PATH = PROJECT_ROOT / "params.yaml"

def _resolve_project_path(value: str | Path) -> str:
    path = Path(value)
    if path.is_absolute():
        return str(path)
    return str((PROJECT_ROOT / path).resolve())

class DataConfig(BaseModel):
    model_config = ConfigDict(protected_namespaces=())
    raw_path: str
    processed_dir: str
    n_samples: int
    test_size: float
    random_state: int

class ModelConfig(BaseModel):
    model_config = ConfigDict(protected_namespaces=())
    max_features: int
    ngram_max: int
    C: float
    max_iter: int
    model_path: str

class EvaluateConfig(BaseModel):
    model_config = ConfigDict(protected_namespaces=())
    metrics_path: str

class Config(BaseModel):
    model_config = ConfigDict(protected_namespaces=())
    data: DataConfig
    model: ModelConfig
    evaluate: EvaluateConfig

def load_config(path: str | Path = PARAMS_PATH) -> Config:
    config_path = Path(path)
    if not config_path.is_absolute():
        config_path = PROJECT_ROOT / config_path

    with open(config_path, encoding="utf-8") as f:
        raw = yaml.safe_load(f)

    cfg = Config(**raw)
    cfg.data.raw_path = _resolve_project_path(cfg.data.raw_path)
    cfg.data.processed_dir = _resolve_project_path(cfg.data.processed_dir)
    cfg.model.model_path = _resolve_project_path(cfg.model.model_path)
    cfg.evaluate.metrics_path = _resolve_project_path(cfg.evaluate.metrics_path)
    return cfg
```

requirements.txt (fixes A7, A8, B4)

```
# Runtime dependencies
scikit-learn==1.5.2
pandas==2.2.3
joblib==1.4.2
PyYAML==6.0.2
pydantic==2.9.2
pydantic-settings==2.6.1
fastapi==0.115.5
uvicorn[standard]==0.32.1
mlflow==2.18.0
dvc==3.55.2
prometheus-fastapi-instrumentator==7.0.0

# Transitive pins. Without these, pip resolves versions that break the
# packages pinned above. Both were found the hard way.
#   pathspec 1.x moved _DIR_MARK; dvc 3.55.2 imports it from the old path.
#   scipy 1.18 dropped the lbfgs iprint option that every sklearn release
#   still sends, producing an OptimizeWarning on every fit.
pathspec==0.12.1
scipy<1.18
numpy<3
```

requirements-serve.txt (new, fixes B12)

```
# Runtime-only dependencies for the Docker image.
# No MLflow, no DVC: neither is used at serving time, and together they
# add a large dependency tree to an image we describe as lean.
scikit-learn==1.5.2
scipy<1.18
numpy<3
pandas==2.2.3
joblib==1.4.2
PyYAML==6.0.2
pydantic==2.9.2
fastapi==0.115.5
uvicorn[standard]==0.32.1
prometheus-fastapi-instrumentator==7.0.0
```

requirements-dev.txt (unchanged)

```
-r requirements.txt
pytest==8.3.3
httpx==0.27.2
ruff==0.7.4
pre-commit==4.0.1
```

.gitignore, the project-specific block (fixes A5, A6, B1)

```
# Virtual environment (this project's is named MLOps, not .env)
MLOps/
.venv/
venv/

# MLflow local tracking store
mlruns/
mlartifacts/

# Data and model artifacts: DVC-tracked, deliberately not in Git
/data/raw/*
/data/processed/*
!/data/raw/.gitkeep
!/data/processed/.gitkeep
/models/*
!/models/.gitkeep

# metrics/metrics.json is deliberately NOT ignored.
# Day 12 wants it visible in PR diffs, and Day 16 declares it with
# cache: false, which means Git owns the file rather than DVC.

# DVC internal cache
/.dvc/cache
/.dvc/tmp
```

Keep the standard Python and IDE sections you already have. Only this block changes.

params.yaml (unchanged; comment moved onto its own line)

```
data:
  raw_path: data/raw/tickets.csv
  processed_dir: data/processed
  n_samples: 3000
  test_size: 0.2
  random_state: 42
model:
  max_features: 5000
  # 1 = unigrams only, 2 = unigrams + bigrams
  ngram_max: 1
  C: 1.0
  max_iter: 1000
  model_path: models/model.joblib
evaluate:
  metrics_path: metrics/metrics.json
```

Your current file has this comment stranded two lines below the key it describes, another artifact of transcribing from the PDF. Harmless to YAML, confusing to a reader.

Part 5 · Troubleshooting

The original table, plus every failure actually encountered on this machine. New rows are marked with the errata ID so you can read the full diagnosis in Part 1.

Symptom	Likely cause	Fix
<code>FileNotFoundError:</code> <code>...\models\train.py</code> in <code>pytest</code>	A test points at a root-level script that does not exist A1	The code is in <code>src/automated_support_ticket_classification/models/</code> . Import the package, do not load files by path.
<code>cannot import name '_DIR_MARK' from 'pathspec.patterns.gitwildmatch'</code>	<code>pathspec</code> 1.x is incompatible with <code>dvc</code> 3.55.2 A7	<code>pip install "pathspec==0.12.1"</code>
output <code>'...'</code> is already tracked by SCM (e.g. Git)	DVC cannot own a file Git already tracks A5	<code>git rm -r --cached data/raw/tickets.csv ...</code> then fix <code>.gitignore</code> . See Repair 3 and 4.
<code>OptimizeWarning: Unknown solver options: iprint</code>	<code>scipy</code> 1.18 dropped an option <code>sklearn</code> still sends A8	<code>pip install "scipy<1.18"</code> . Upgrading <code>scikit-learn</code> does not help; every release through 1.9 still sends it.
Wall of <code>PydanticDeprecatedSince20</code> on every <code>pytest</code> run	MLflow 2.18 uses <code>pydantic</code> v1 validators A9	Add the <code>filterwarnings</code> block to <code>[tool.pytest.ini_options]</code> .
Ruff reports hundreds of errors in files you never wrote	It is linting your virtual environment B13	Add <code>ML0ps</code> to <code>[tool.ruff] exclude</code> . Ruff only auto-excludes conventional names like <code>.venv</code> .
<code>metrics.json</code> never shows up in a commit	<code>metrics/*.json</code> is in <code>.gitignore</code> A6	Remove that line. Day 16's <code>cache: false</code> requires Git to own the file.
<code>git add data models</code> exits 1	Those directories are ignored B1	Ignore the contents, not the directory, and keep <code>.gitkeep</code> negations. See Repair 3.
<code>FileNotFoundError:</code> <code>'params.yaml'</code>	The config path was relative to the working directory B5	Anchor with <code>Path(__file__).resolve().parents[2]</code> . Already fixed in your repo.
<code>UserWarning: Field "model_path" ... protected namespace "model_"</code>	<code>Pydantic</code> v2 reserves the <code>model_</code> prefix B6	<code>model_config = ConfigDict(protected_namespaces=())</code> on each model.
<code>TypeError: Object of type float64 is not JSON serializable</code>	A numpy scalar reached <code>json.dump</code> B7	Cast with <code>float(...)</code> where the metric is computed, not where it is written.
<code>ModuleNotFoundError: No module named 'ticket_triage'</code>	You followed the other PDF for one step C4	The package is <code>automated_support_ticket_classification</code> . Archive the Ticket-Triage PDF.
<code>pip install -e .</code> succeeds but the import still fails	No <code>__init__.py</code> , so zero packages were found B2	Create <code>src/automated_support_ticket_classification/__init__.py</code> , then reinstall.

<code>Activate.ps1</code> is blocked	PowerShell execution policy	<code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code> , or use <code>activate.bat</code> .
pytest cannot import from <code>src</code>	<code>src</code> not on the path	The <code>pythonpath = ["src"]</code> block in <code>[tool.pytest.ini_options]</code> fixes it.
<code>mlflow ui</code> shows nothing	Wrong folder, or no runs yet	Train first, and launch <code>mlflow ui</code> from the repo root so it finds <code>./mlruns</code> .
<code>dvc repro</code> reruns everything	A dep or param hash changed	Expected after edits. Unchanged stages stay cached. If it reruns with no edits, a declared dep is being rewritten by another stage.
Docker: connection refused	uvicorn bound to 127.0.0.1	Bind <code>0.0.0.0</code> (the supplied <code>CMD</code> does) and map <code>-p 8000:8000</code> .
Docker build is very slow or the image is huge	<code>.dockerignore</code> is missing your venv	Add <code>MLOps</code> to <code>.dockerignore</code> , otherwise the whole environment is sent as build context.
Port 8000 already in use	Another process holds it	Stop it, or map a different host port: <code>-p 8001:8000</code> .
Render: first request very slow	Free instance cold start	Expected on the free tier. The instance sleeps when idle. Mention it rather than apologising for it.
<code>B905 zip() without an explicit strict=</code>	The guide's Day 20 code violates a rule Day 4 enables B16	Add <code>strict=True</code> . Do not suppress the rule; the sequences genuinely must match in length.
<code>dvc repro</code> fails with <code>ModuleNotFoundError</code>	DVC spawned a different <code>python</code>	<code>dvc.yaml</code> hardcodes <code>python</code> , so run it from the activated environment or put the venv first on PATH.
Only four classes in <code>dvc metrics show</code>	Transcription dropped a template block D1	Restore <code>shipping</code> and move the technical templates out of <code>account</code> . See Group D.
CI fails on the format check	Code is not ruff-formatted	Run <code>ruff format</code> locally and commit. Green locally is the only reliable predictor of green in CI.

Part 6 · Verification matrix

One row per day from here to the end. If the result column does not match, stop and fix it before moving on; every day depends on the one before.

Day	Run	Expected
Repair	<code>pytest</code>	13 passed , no OptimizeWarning
Repair	<code>ruff check . ; ruff format --check .</code>	All checks passed, 0 to reformat
Repair	<code>dvc --version</code>	3.55.2 , no traceback
Repair	<code>git ls-files data models</code>	only <code>.gitkeep</code> entries
14	<code>python -m automated_support_ticket_classification.models.train</code>	a second run appears in <code>mlflow ui</code>
15	<code>dvc remote list</code>	<code>localremote ../dvc-storage</code>
16	<code>dvc dag</code>	generate to preprocess to train to evaluate
17	<code>dvc repro</code>	four stages run, <code>dvc.lock</code> written
17	<code>dvc metrics show</code>	accuracy and f1_macro printed
18	<code>dvc status -c</code>	Data and pipelines are up to date.
19	<code>python -c "...TicketRequest(text='hi')"</code>	<code>text='hi'</code>
20	<code>Invoke-RestMethod http://127.0.0.1:8000/health</code>	<code>ok</code>
21	POST to <code>/predict</code>	label, confidence, all_scores
22	<code>pytest</code>	13 passed (8 in the original, plus B11's test and two more)
23	<code>docker build -t</code>	exporting to image ... DONE , 711 MB, context 3.26 kB
24	<code>docker compose up --build</code>	<code>/health</code> answers on port 8000
25	<code>git push</code>	<code>lint-and-test</code> green on first run
26	<code>git push</code>	both jobs green, <code>docker-build</code> after tests
27	open <code>:8000/metrics</code> and <code>:9090</code>	counters appear once traffic exists
28	open the Render URL	<code>/docs</code> serves a live prediction
29	run Smoke Test from the Actions tab	Health status code: 200
30	<code>git push origin v1.0.0</code>	tag visible, badge green, demo link live

Final checklist

- ☐ Repo initialised with README, LICENSE, `.gitignore`
- ☐ Virtual environment, pinned dependencies including transitive pins, editable install
- ☐ ruff clean: `check` and `format --check` both pass
- ☐ `params.yaml` plus typed config, paths anchored to the project root
- ☐ Reproducible data generation and preprocessing
- ☐ Training with an sklearn Pipeline and MLflow tracking
- ☐ Evaluation writing a Git-tracked `metrics.json`
- ☐ Unit and API tests passing, including the whitespace-only case

- ☐ DVC initialised, pipeline defined, dvc.lock committed, artifacts pushed
- ☐ Data artifacts owned by DVC and by nothing else
- ☐ FastAPI service with /health, /predict, /metrics, sharing clean_text with training
- ☐ Multi-stage Dockerfile building from requirements-serve.txt
- ☐ GitHub Actions CI green (format, lint, test, docker build) with a badge
- ☐ Prometheus scraping the service locally
- ☐ Deployed to Render with a live public URL
- ☐ Auto-deploy plus scheduled smoke test working
- ☐ Polished README with real links, no placeholders, and a v1.0.0 tag

Part 7 · Interview preparation

The original twelve questions, kept because they are good, plus five the repair sprint earned you. The new ones are stronger material than the originals, because they describe something that actually happened to you rather than something you read.

Core questions

Q: What is MLOps in one sentence?

A: Applying software engineering and DevOps discipline, meaning versioning, testing, CI/CD, deployment and monitoring, to the full machine learning lifecycle so models ship reliably and stay healthy in production.

Q: Walk me through this project's architecture.

A: A DVC pipeline generates synthetic ticket data, preprocesses and splits it, trains a TF-IDF plus Logistic Regression pipeline tracked in MLflow, and evaluates it into a versioned metrics file. The saved sklearn Pipeline is served by FastAPI, containerised with Docker, linted, tested and built by GitHub Actions, deployed to Render, and monitored by Prometheus.

Q: How is this reproducible?

A: Code is pinned by Git, data by a seed and by DVC, hyperparameters by params.yaml, the environment by pinned requirements plus a lock file, and the exact run by the hashes in dvc.lock. The same commit reproduces the same model. I would add that pinning direct dependencies is not the same as pinning the graph, which I learned the hard way on this project.

Q: Why classical ML instead of a transformer?

A: The task is clean text classification where a linear model is fast, cheap, interpretable, needs no GPU, and scores well. The engineering around the model is what this project demonstrates, and a model that trains in seconds keeps the iteration loop tight.

Q: How do you prevent data leakage?

A: Split before fitting anything, and keep every transform inside an sklearn Pipeline that is fit only on the training fold. The Pipeline also guarantees the identical transform at inference, which is the other half of the same problem.

Q: How does the model reach production?

A: It is baked into the Docker image at build time, so the container is self-contained with no external storage dependency. CI builds the image and Render health-gates the rollout. The trade is that shipping a new model means rebuilding the image; a model registry is the alternative and it is on my improvements list for exactly that reason.

Q: What do you monitor?

A: Operationally, the RED signals: request rate, error rate and duration, via prometheus-fastapi-instrumentator scraped by Prometheus. For ML specifically you would add input and prediction drift and data quality checks, which is a separate concern from service health and needs separate tooling such as Evidently.

Q: How would you retrain on new data?

A: Point the generate stage at the new source, run `dvc repro` so only changed stages rerun, review the metrics diff in the pull request, and merge if it improved. The merge triggers the deploy pipeline.

Q: Continuous delivery versus continuous deployment?

A: Delivery keeps main always releasable with a manual approval to ship. Deployment ships every passing merge automatically. This project does deployment, with a scheduled smoke test as the verification step.

Q: How would you scale this?

A: The service is stateless, so run multiple replicas behind a load balancer. Move the model out of the image into a registry or object storage so replicas share one artifact and can be updated without a rebuild, and add caching if the same tickets recur.

Q: Where would Kubernetes fit?

A: When you need multi-service orchestration, autoscaling, rolling updates and self-healing beyond what a PaaS gives you. It is the natural next step once traffic and complexity grow, not before.

Q: What are the weaknesses of this design?

A: Synthetic data, a single-node deploy, no model registry, no drift detection, and a cold-starting free tier. Each is a deliberate scope decision with a named next step rather than an oversight.

Questions the repair sprint earned you

These describe real debugging you did. Interviewers can tell the difference, and this is the material that separates a tutorial-follower from an engineer.

Q: Tell me about a dependency problem you had to debug.

A: My DVC install was pinned at 3.55.2 but every command died with an ImportError on a private symbol. The symbol, `_DIR_MARK`, had moved between minor versions of pathspec, a transitive dependency I had never pinned, from `patterns/gitwildmatch.py` into `patterns/gitignore/spec.py`. I confirmed it by reading both versions' source rather than guessing, pinned pathspec to 0.12.1, and added a lock file. The lesson was that pinning my direct dependencies gave me false confidence: reproducibility is a property of the whole graph.

Q: Describe a time the obvious fix was wrong.

A: Every model fit emitted an `OptimizeWarning` about an unknown solver option, `iprint`, from a scikit-learn file. The instinct is to upgrade scikit-learn. I checked the source of every release from 1.5 through 1.9 and all of them still send that option, so upgrading would have changed nothing. The option had been removed on the SciPy side, between 1.17 and 1.18. Pinning SciPy fixed it. The general lesson is to find the layer that actually changed before you change anything.

Q: Have you ever had two tools fight over the same resource?

A: Yes. My data CSVs were committed to Git before I introduced DVC, and `dvc repro` refused to run: an output cannot be tracked by SCM and by DVC at once. That is correct behaviour rather than a bug, because two version control systems cannot both own one file's history. I untracked them with `git rm --cached`, which leaves the files on disk, and fixed the gitignore rules that let them in. The subtlety in those rules is that Git will not descend into an ignored directory, so you have to ignore the contents and negate the placeholder, not ignore the directory itself.

Q: How do you keep a production container small?

A: Multi-stage build, slim base image, and a separate runtime requirements file. My training code tracks experiments in MLflow, but MLflow has no role at serving time, so the image installs `requirements-serve.txt`, which omits MLflow and DVC entirely. The training module imports MLflow inside a try/except and degrades gracefully, so the same file works in both places. Worth noting: I originally had an env flag to disable MLflow, which stopped it being used but not being installed. The flag was treating the symptom.

Q: What is train/serve skew and did you hit it?

A: It is when the transformation applied at inference differs from the one applied at training, so the model sees a distribution it was never fit on. I had a mild case: training normalised text with a `clean_text` function that lowercases and collapses whitespace, while the API endpoint did its own `strip().lower()`. With TF-IDF's default tokenizer the practical impact was near zero, but having two independent definitions of "clean" guarantees they diverge as preprocessing grows. I made the API import the same function. The structural fix is what the sklearn Pipeline already does for the model: one artifact, one transform, no second implementation to drift.

Closing note

The defects in this document are not a sign that the original guide was bad. Every long technical document accumulates them, and finding them is the job. What matters is that you hit a failure, read the traceback rather than the tutorial, and fixed the cause instead of the symptom. Say that in an interview and mean it, because on this project it is true.